

Ex 7. Using Snort, perform real-time traffic analysis and packet logging.

Objective

To analyze live network traffic in real time and log packets using **Snort** (an open-source Intrusion Detection System).

Requirements

- **OS:** Linux (Ubuntu/Kali preferred)
- **Privileges:** Root/Administrator access
- **Software:** Snort installed (verify using `snort -V`)

Step 1: Verify Snort Installation

Open a terminal and type:

```
snort -V
```

You should see version details confirming that Snort is installed.

Step 2: Identify Your Network Interface

Find available interfaces:

```
ip link show
```

or

```
snort -W
```

or

```
ifconfig
```

Example: `eth0`, `wlan0`, or `ens33`

Step 3: Run Snort in Packet Sniffer Mode

This mode allows you to see live traffic without saving it.

```
sudo snort -v -i eth0
```

- `-v` → verbose mode (prints packets to screen)
- `-i eth0` → capture from the `eth0` interface

Observation:

You'll see source/destination IPs, protocols, and packet details in real time.

Step 4: Run Snort in Packet Logger Mode

Now capture and save packets for later analysis.

```
sudo snort -dev -l /var/log/snort/
```

- `-d` → show application layer data
- `-e` → show link-layer headers
- `-v` → verbose mode
- `-l` → specify log directory

Perform some activity (like browsing or pinging):

```
ping www.google.com
```

```
Stop Snort (Ctrl + C).
```

Check Logs:

```
ls /var/log/snort
```

Snort stores logs in binary format (snort.log.*).

You can analyze them using:

```
sudo tcpdump -r /var/log/snort/snort.log.*
```

Step 5: Run Snort in Real-Time IDS Mode

This mode analyzes traffic live and triggers alerts based on rules.

```
sudo snort -A console -q -c /etc/snort/snort.conf -i eth0
```

- -A console → display alerts on the terminal
- -q → quiet mode (hide packet details)
- -c → specify configuration file
- -i → interface name

Generate some traffic (open a web page or ping).

Snort will print alerts like:

```
[**] [1:1000001:0] ICMP Packet Detected [**]  
[Classification: Misc Activity] [Priority: 3]
```

Step 6: Create and Test a Custom Rule

Let's make Snort alert when it detects ICMP (ping) packets.

Edit the local rules file:

```
sudo nano /etc/snort/rules/local.rules
```

Add this rule:

```
alert icmp any any -> any any (msg:"ICMP Packet Detected"; sid:1000001; rev:1;)
```

Save and exit, then include this rule in your config (snort.conf):

```
include /etc/snort/rules/local.rules
```

Run Snort:

```
sudo snort -A console -q -c /etc/snort/snort.conf -i eth0
```

Now, from another terminal:

```
ping -c 3 8.8.8.8
```

You'll see alerts like:

```
[**] [1:1000001:1] ICMP Packet Detected [**]  
[Priority: 3] {ICMP} 192.168.1.5 -> 8.8.8.8
```

Step 7: Analyze Logs

Check the Snort log directory:

```
cd /var/log/snort
```

```
sudo cat alert
```

You'll find alert messages with timestamps, source/destination IPs, and priorities.

Output

- **Sniffer Mode:** Displays live packets
- **Logger Mode:** Saves packets to files
- **IDS Mode:** Detects threats and triggers alerts
- **Custom Rules:** Allow personalized detection signatures

Example Report Format

Step	Mode	Command	Observation
------	------	---------	-------------

1	Sniffer	snort -v -i eth0	Displayed live packets
2	Logger	snort -dev -l /var/log/snort/	Saved packets to log file
3	IDS	snort -A console -q -c /etc/snort/snort.conf -i eth0	Generated real-time alerts
4	Custom Rule	alert icmp any any -> any any ...	Triggered alert on ping packets

Optional: View Logs in Wireshark

You can open the .log file directly in Wireshark:

wireshark /var/log/snort/snort.log.*

This helps you visualize packet structures and flow.

```
(kali@kali)~$ sudo snort -v -i eth0
-----
o")~   Snort++ 3.1.82.0
-----
Network Policy : policy id 0 :
-----
Inspection Policy : policy id 0 :
-----
pcap DAQ configured to passive.
-----
host_cache
  memcap: 33554432 bytes
Commencing packet processing
++ [0] eth0
Instance 0 daq pool size: 256
Instance 0 daq batch size: 64
█
```

After Some Time (2 or 3 minutes) Press **Ctrl + C**

```
(kali@kali)~$ sudo snort -v -i eth0
[sudo] password for kali:
-----
o")~   Snort++ 3.1.82.0
-----
Network Policy : policy id 0 :
-----
Inspection Policy : policy id 0 :
-----
pcap DAQ configured to passive.
-----
host_cache
  memcap: 33554432 bytes
Commencing packet processing
++ [0] eth0
Instance 0 daq pool size: 256
Instance 0 daq batch size: 64
^C** caught int signal
== stopping
-- [0] eth0
-----
Packet Statistics
-----
```

```

-----
Packet Statistics
-----
daq
    received: 2261
    analyzed: 2261
    allow: 2261
    rx_bytes: 1370572
-----
codec
    total: 2261          (100.000%)
    discards: 891        ( 39.407%)
    arp: 23              (  1.017%)
    eth: 2261            (100.000%)
    icmp6: 6             (  0.265%)
    igmp: 5              (  0.221%)
    ipv4: 2230           ( 98.629%)
    ipv6: 8              (  0.354%)
    ipv6_hop_opts: 5     (  0.221%)
    tcp: 786             ( 34.763%)
    udp: 1320            ( 58.381%)
-----
Module Statistics
-----
detection
    analyzed: 2261
-----
tcp
    bad_tcp4_checksum: 431
-----
udp
    bad_udp4_checksum: 339
-----
Summary Statistics
-----
process
    signals: 1
-----
timing
    runtime: 00:01:13
    seconds: 73.165391
    pkts/sec: 31
o")~  Snort exiting

```

— this means **Snort is now successfully capturing live network traffic** from your eth0 interface!

Understanding the Output

1. DAQ (Data Acquisition Layer)

pcap DAQ configured to passive.

- Snort is using the **pcap** driver to capture packets (same as Wireshark).
- **Passive mode** = Snort only *monitors* packets, not modifies them — good for lab use.

2. Packet Statistics

received: 2261

analyzed: 2261

allow: 2261

rx_bytes: 1370572

- Snort captured **2,261 packets** in about **73 seconds**.
- That's roughly **31 packets/sec**, showing **active traffic** on your bridge connection.
This confirms your bridged mode and network setup are working perfectly.

3. Protocol Breakdown

Protocol Count Description

ARP	23	Address Resolution Protocol — used for IP-to-MAC mapping
ICMP6	6	IPv6 ping packets
IGMP	5	IPv6 group management (used in multicast)
IPv4	2230	Regular IPv4 traffic
IPv6	8	Some IPv6 packets detected
TCP	786	Common web/email traffic
UDP	1320	Likely DNS, DHCP, or streaming data

So you're seeing **real network data**: ARP, TCP, UDP, ICMP, etc.

4. Checksum Errors

bad_tcp4_checksum: 431

bad_udp4_checksum: 339

- These are **not a problem** — they often occur when:
 - Packets are captured **before checksums are computed** (hardware offloading).
 - You're in a **virtual machine** using bridged adapters.
- You can safely ignore them in this setup.

5. Timing Summary

runtime: 00:01:13

pkts/sec: 31

- Snort analyzed traffic in real time — everything is functioning as expected.

Result

I have successfully:

1. Configured **Snort** in real-time monitoring mode.
2. Switched to **bridged mode** or NAT and obtained network visibility.
3. Captured and analyzed **live packets** on my network.

Snort is **fully operational** on your system.